When Index Tuning Advice Backfires: An Experimental Study of Access-Path Selection in PostgreSQL

Md. Imtiaj Alam Sajin[a,*], Ashraf Uddin[a]

[a]*Department of Computer Science, American International University-Bangladesh, 408/1 Kuratoli, Khilkhet, Dhaka 1229, Bangladesh*

Abstract

Selecting the wrong index can slow a query by orders of magnitude, and a substantial body of practitioner guidance exists to help database administrators avoid that outcome. We ask whether that guidance survives measurement. We construct a controlled benchmark in which value skew, predicate correlation and physical clustering are varied independently, with ground-truth cardinality computed from the generated data rather than obtained from the database, and report 14,296 measured query executions against PostgreSQL 17.1 across seven table scales, eleven dataset configurations, ten index configurations and seven cost-model settings. Our first result is reassuring: under a conventional index set the planner selects a near-optimal access path, erring materially for only 2.3% of queries at a median penalty of 1.3%. Our remaining results are not. We evaluate three widely recommended practices and find that each degrades performance under identifiable conditions. Extended statistics, the documented remedy for correlated predicates, improve cardinality estimates by up to 108× while making the affected queries up to 2.7× slower, because the correction is not propagated to the `BitmapAnd` node that consumes it. Reducing `random_page_cost` below its default on solid-state storage, advice repeated widely, slows our workload by 2.3×; the default lies within 6% of optimal. Adding a BRIN index alongside a B-tree on the same column raises misselection from 2.3% to 73.3%,

*Corresponding author.

Email addresses: 26-94090-2@student.aiub.edu (Md. Imtiaj Alam Sajin), dr.ashraf@aiub.edu (Ashraf Uddin)

with individual queries degrading by up to 194×. We further show that this last effect has two distinct boundaries rather than one: its frequency collapses when the heap exceeds `shared_buffers`, whereas its worst-case severity doubles at that same point and subsides only when queries begin incurring physical reads several-fold further out. A two-point scale comparison would have supported the opposite conclusion. Notably, cardinality misestimation does not explain the misselection we observe: 98.2% of misselected queries had accurate row estimates. All code, raw measurements and analysis are published.

Keywords: query optimisation, access path selection, index selection, cardinality estimation, cost model, PostgreSQL, performance benchmarking

---

1. Introduction

The access-path decision, whether to read a table sequentially or through one of several available indexes, is among the most consequential choices a query optimiser makes. It was formalised in the System R optimiser (Selinger et al., 1979) and the same framework, estimate cardinality and then price each candidate path, underpins PostgreSQL today. Depending on that single decision, the same query can differ by two orders of magnitude in execution time.

Because the decision matters and is opaque to the user, a large body of tuning guidance has accumulated around it. Administrators are advised to create extended statistics when predicate columns are correlated (PostgreSQL Global Development Group, 2024b), to lower `random_page_cost` when running on solid-state storage (PostgreSQL Global Development Group, 2024d), and to add Block Range Index (BRIN) structures on naturally ordered columns because they are small and inexpensive to build (PostgreSQL Global Development Group, 2024a). This guidance appears in official documentation, vendor engineering material and community forums. It is generally justified by a mechanism argument, an explanation of why the change should help, rather than by workload-level measurement.

Mechanism arguments are seductive because they are usually correct as far as they go. Extended statistics really do improve cardinality estimates. Random reads on flash storage really are cheaper relative to sequential reads than they were on magnetic disks (Graefe, 2009; Lee et al., 2009). BRIN indexes really are orders of magnitude smaller than B-trees. The question

this paper asks is whether being correct about the named quantity is sufficient to improve the outcome that actually matters, namely the runtime of the workload.

1.1. Research questions

RQ1 How often does the PostgreSQL planner select a slower access path than one available to it, and what does that cost?

RQ2 Which data properties drive misselection: value skew, correlation between predicate columns, or physical clustering?

RQ3 Is cardinality misestimation the cause, as the query optimisation literature would predict?

RQ4 Do the three tuning practices named above improve access-path selection?

RQ5 Under what conditions do the observed effects cease to hold?

RQ5 deserves emphasis. Empirical database studies frequently report that an effect exists without establishing the conditions under which it disappears. That omission is consequential: a practitioner cannot tell whether a published finding applies to their deployment, and a subsequent study measuring at a different scale may appear to contradict the original when in fact both are correct within their respective regimes. We show in Section 5.6 that this is not hypothetical, and that our own two-point comparison would have yielded a conclusion the intermediate measurements refute.

1.2. Contributions

1. A controlled benchmark in which skew, predicate correlation and physical clustering vary independently, with exact ground-truth cardinality for every predicate, obtained by counting the generated data rather than by querying the system under test.
2. A quantification of PostgreSQL 17's access-path selection quality, which we find to be near optimal under a conventional index set (Section 5.1).
3. Evidence that cardinality misestimation does not explain single-table access-path misselection, in contrast to the established result for join ordering (Leis et al., 2015) (Section 5.2).

4. Three measured cases in which recommended tuning practice degrades performance, each traced to a mechanism visible in the query plan (Sections 5.3–5.5).
5. A characterisation of the scale boundaries of the largest effect, showing that its frequency and its severity collapse at two different and separately identifiable system boundaries (Section 5.6).
6. A fully reproducible artifact: seeded data generation, resumable measurement, and one-command regeneration of every figure (Sajin, 2026).

2. Background

2.1. Access-path selection

Given a predicate over a single relation, a cost-based optimiser enumerates candidate access paths, a sequential scan plus one path per usable index, estimates the number of rows each will produce, and assigns each a cost (Selinger et al., 1979; Chaudhuri, 1998; Graefe, 1993). In PostgreSQL, cost is expressed in abstract units anchored to `seq_page_cost`, with `random_page_cost` expressing how much more expensive a randomly fetched page is assumed to be relative to a sequentially fetched one (PostgreSQL Global Development Group, 2024e). The default ratio of 4:1 encodes an assumption about magnetic disk behaviour.

A bitmap scan occupies an intermediate position between an index scan and a sequential scan. The index is scanned to build a bitmap of matching tuple identifiers or block ranges, the bitmap is sorted into physical order, and the heap is then read in that order, converting random access into something closer to sequential access. The cost of a bitmap heap scan is therefore dominated by the number of heap pages the optimiser expects to fetch, priced using `random_page_cost` interpolated toward `seq_page_cost` as the fraction of the table touched grows.

2.2. Index structures

A B-tree supports both equality and range predicates and yields exact tuple pointers (Comer, 1979; Graefe, 2011; Silberschatz et al., 2019). A hash index supports equality only. A BRIN index stores summary information, typically minimum and maximum values, for each range of physical blocks rather than for each tuple (PostgreSQL Global Development Group, 2024a). It is consequently very small, often by three orders of magnitude, but it returns *candidate block ranges* rather than rows, so every tuple in a candidate

range must be rechecked against the predicate. Its usefulness therefore depends entirely on the correlation between indexed value and physical row position. When correlation is high, few block ranges qualify and the recheck cost is modest; when correlation is low, most ranges overlap the predicate and the scan degenerates toward a full table scan with additional recheck overhead.

BRIN therefore exemplifies the specialisation trend (Stonebraker and Cetintemel, 2005): it abandons the generality of a B-tree in exchange for a size reduction of three orders of magnitude, and is correspondingly effective only on the narrow class of workloads whose physical layout suits it. That trade is central to our results, because the optimiser must recognise when the specialised structure does not apply.

2.3. Cardinality estimation

PostgreSQL estimates the selectivity of a conjunctive predicate by multiplying per-column selectivities, which assumes column independence (PostgreSQL Global Development Group, 2024c). Where columns are correlated, the estimate can be wrong by orders of magnitude, and the error is systematically an underestimate for positively correlated attributes. Extended statistics objects, created with `CREATE STATISTICS`, capture multivariate distributional information and are the documented remedy (PostgreSQL Global Development Group, 2024b).

2.4. Buffer management

PostgreSQL maintains its own buffer pool, sized by `shared_buffers`, above the operating system page cache (Effelsberg and Haerder, 1984). A logical read satisfied from the buffer pool costs a pointer dereference; one that misses requires a system call and a memory copy even when the operating system holds the page, and physical I/O when it does not. This layering matters for our results because a working set may exceed `shared_buffers` while remaining entirely within the page cache, producing a regime that is neither fully resident nor genuinely I/O bound.

3. Related Work

Optimiser quality and cardinality estimation. Leis et al. (2015, 2018) introduced the Join Order Benchmark and established that cardinality misestimation, rather than cost-model calibration or plan enumeration, dominates

plan quality for join queries. Their analysis showed estimation errors compounding multiplicatively through join trees, consistent with earlier theoretical work on error propagation (Ioannidis and Christodoulakis, 1991). Their focus is join ordering; the single-relation access-path decision is not isolated. Our results are complementary and, on one point, opposed: for the access-path decision we find misestimation is *not* the primary driver of the miss-election we observe (Section 5.2). This is not a contradiction. Access-path selection has a small candidate set and no compounding, so it is plausible that a different failure mode dominates.

Moerkotte et al. (2009) introduced q-error, the symmetric ratio between estimated and actual cardinality, and proved bounds relating it to plan cost degradation. We adopt it unchanged. Han et al. (2021) provide a comprehensive benchmark of cardinality estimators and likewise observe that estimation accuracy and end-to-end plan quality are imperfectly coupled, a theme our extended-statistics result develops in a specific mechanistic case. Negi et al. (2021) make the same observation constructively, training estimators against a plan-cost objective rather than an accuracy objective on the grounds that the two diverge.

Cost models. Wu et al. (2013) examined whether optimiser cost models can predict execution time and found them usable only after calibration. More recent work replaces the analytic cost model with a learned one (Marcus et al., 2019, 2021). That line of work implicitly accepts that the shipped cost model is miscalibrated; our contribution is to measure where, for one specific decision, and to quantify what the miscalibration costs relative to simply leaving the defaults alone. Lan et al. (2021) survey the area and note the scarcity of controlled studies isolating individual optimiser components.

Index selection and physical design. A long line of work addresses which indexes to create (Chaudhuri and Narasayya, 1997), recently evaluated comparatively by Kossmann et al. (2020). That literature asks which index set minimises cost under a workload, assuming the optimiser will use the chosen indexes appropriately. We examine the complementary and apparently unexamined question of whether adding an index can degrade performance by changing which paths the optimiser considers, and we answer it affirmatively in Section 5.5.

Index structure benchmarking. Wongkham et al. (2022) provide a methodological model for rigorous index evaluation, in particular the practice of

measuring each index design in isolation rather than inferring behaviour from composite configurations, which we adopt. Their subject, learned indexes (Kraska et al., 2018), differs from ours, but the experimental discipline transfers directly.

Benchmarking methodology. Raasveldt et al. (2018) catalogue pitfalls in database performance testing, including uncontrolled caching, single-run reporting, and measurement instruments whose overhead depends on the plan being measured. The last is directly relevant: we show in Section 4.5 that the obvious instrument for this study introduces exactly the plan-dependent bias they warn against, and we describe the configuration that avoids it. Boncz et al. (2013) demonstrate the value of analysing why a benchmark produces the results it does rather than reporting them at face value, an approach we follow by tracing each finding to plan-level evidence.

System-level reports. The behaviour we report for BRIN is adjacent to a difficulty acknowledged on the PostgreSQL development mailing list (PostgreSQL Hackers Mailing List, 2019), where BRIN cost estimation is discussed as unreliable. That discussion concerns the opposite direction, BRIN being faster but not selected. We are not aware of a systematic quantification of the direction reported here, nor of its scale boundaries.

4. Experimental Design

4.1. Metrics

We use q-error (Moerkotte et al., 2009) for estimation accuracy:

$$q(\hat{n}, n) = \frac{\max(\hat{n}, n)}{\min(\hat{n}, n)}, \tag{1}$$

where $\hat{n}$ is the estimate and n the true row count, both floored at one. A value of 1.0 denotes a perfect estimate. The symmetric ratio form is used because a tenfold underestimate and a tenfold overestimate are comparably damaging to plan selection.

To measure the cost of the decision itself we define *access-path regret*:

$$R(q) = \frac{T_{\text{chosen}}(q)}{\min_{a \in A} T_a(q)}, \tag{2}$$

where A is the set of index configurations and T_a is median execution time under configuration a. $R = 1$ indicates the planner selected the best available path; $R = 4$ indicates the query took four times longer than necessary. Because the denominator is the fastest configuration *measured* rather than a proven optimum, reported regret is a **lower bound** on the true penalty.

We define *misselection* as $R > 1.2$. The threshold prevents run-to-run variation being counted as an optimiser error. It is a judgement call; we report full distributions so readers may apply a different one.

4.2. Controlled data generation

Real datasets do not permit varying one property while holding others constant, so all data is generated. Three factors vary independently around a uniform, independent baseline, so that each effect is attributable (Table 1).

Table 1: Controlled factors. Eleven dataset configurations result, each varying a single factor from the baseline.

Factor	Levels	Controls
`zipf_s`	0.0, 0.5, 1.0, 1.5	Value skew of the indexed column
`dep_strength`	0.0, 0.25, 0.5, 0.75, 1.0	Strength of functional dependency $a \to b$
`physical_corr`	0.0, 0.5, 0.95, 1.0	Correlation of value order with row order

Skew is produced by sampling from a truncated Zipfian distribution over a fixed domain, so that the number of distinct values remains constant and only the shape of the distribution changes. The functional dependency is induced by setting $b = g(a)$ with probability `dep_strength` and drawing b uniformly otherwise, giving a dependency of tunable strength, which is precisely the structure the `dependencies` extended-statistics kind is designed to capture. Physical correlation is produced by sorting a timestamp column and then permuting a controlled fraction of positions.

Domain sizing is load-bearing. With 10^6 rows and 100 distinct values in each of a and b, an independent conjunction `a = va AND b = vb` returns approximately 100 rows, exactly what the independence assumption predicts, which serves as the control. Under a full functional dependency the same predicate returns all 10^4 rows sharing `a = va` while the estimate remains at 100. That is a q-error of 100 at 1% selectivity, which is the region in which index and sequential access compete most closely.

4.3. Query families

Four families are used. Every predicate's true cardinality is computed by counting the generated arrays directly, never by asking the system under test:

- **Equality** on the skewed column. Candidate paths: hash, B-tree, BRIN, sequential scan.
- **Range** on the skewed column. Hash cannot serve this, which tests fallback behaviour.
- **Range on the physically clustered column**, BRIN's most favourable case.
- **Conjunctive** on the correlated pair, in dependent and independent variants, the latter serving as a control.

Predicate constants are chosen by a two-pointer sweep over the distinct values and their cumulative counts, selecting the contiguous value range whose total is nearest a target selectivity. An earlier implementation anchored a fixed-width window at the median row position; on a Zipfian column the median row falls inside a single very frequent value, so the bounds collapsed and the predicate returned that value's entire frequency regardless of the target. We report this because the failure was silent: recorded row counts remained exact, so no metric was wrong, but the intended selectivity sweep did not occur on skewed data. Affected measurements were discarded and recollected.

4.4. Isolating access paths

PostgreSQL provides no mechanism to hide one index from the optimiser while leaving others visible. We therefore measure each candidate path by building that index *alone* and executing the full query set, following Wongkham et al. (2022). Two further configurations build every index simultaneously, one including BRIN and one excluding it; these reproduce realistic deployments and are where the optimiser's free choice is observed. Regret per Equation (2) is the ratio between a free-choice configuration and the best single-index configuration for the same query.

4.5. Measurement protocol

Every timed execution used `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)`, reading the top-level `Execution Time`. Three considerations motivate this choice, each bearing on validity (Raasveldt et al., 2018):

1. `ANALYZE` executes server-side and discards the result set, so client transfer never enters the measurement.
2. `TIMING OFF` disables per-node instrumentation. With timing enabled, PostgreSQL reads the system clock twice per emitted row. That overhead is *plan-dependent*: a sequential scan emitting 10^6 rows pays it 2×10^6 times while an index scan emitting 10^4 rows pays it 2×10^4 times. Using the instrumented instrument would therefore bias the comparison toward plans emitting fewer rows, which is precisely the comparison being made.
3. Applying one instrument uniformly means any residual overhead is common mode and cancels in the ratio defining regret.

Each query executed seven times; the first was discarded as cache-warming and the median of the remaining six reported, with all individual runs retained for independent verification of variance claims. Parallelism was disabled (`max_parallel_workers_per_gather = 0`) so that it could not confound the serial access-path decision, and just-in-time compilation was disabled because it engages on cost thresholds and would therefore fire inconsistently across configurations.

4.6. Validity checks

Nineteen automated checks execute before any measurement is trusted. Zipfian skew concentrates the top ten values from 0.3% of rows at $s = 0$ to 76.8% at $s = 1.5$. Realised dependency strength lands within 0.005 of the requested value at every level. Realised physical rank correlation is 0.005, 0.497, 0.949 and 1.000 for the four settings and is independently confirmed against PostgreSQL's own `pg_stats` correlation estimate at run time, which validates the generator against a measurement we do not control. Every predicate's row count matches a direct recount of the generated arrays. Identical seeds produce identical data.

Table 2: Experimental environment. Complete settings are captured automatically per run in the artifact (Sajin, 2026).

Component	Specification
CPU	Intel Core i5-12400, 6 cores / 12 threads
Memory	23.8 GB
Storage	SSD, dedicated tablespace on a device separate from the OS
DBMS	PostgreSQL 17.1, x86_64, MSVC 19.41
`shared_buffers`	128 MB
`work_mem`	4 MB
`random_page_cost`	4.0 (default); swept 4.0 to 1.0
Table scales	10^6 to 10^7 rows (seven scales)
Measured executions	14,296

4.7. Environment

5. Results

5.1. RQ1: baseline optimiser quality

Under a conventional index set, that is B-tree and hash indexes without BRIN, the optimiser performs well. At 10^6 rows it selected the best available access path for 97.7% of queries, and where it erred the median penalty was 1.3% (Table 3).

Table 3: Access-path selection quality under a conventional index set.

Scale	Queries	Misselection	Median R	p90 R	Max R
10^6	130	2.3%	1.013	1.13	1.39
10^7	155	32.3%	1.043	1.52	3.70

This positive result frames what follows. PostgreSQL's access-path selection is not broken, so the failures reported below are specific rather than symptomatic of a generally poor optimiser. It also establishes the baseline against which the tuning practices must be judged: any practice must improve on 2.3% misselection to be worth adopting.

5.2. RQ3: misestimation does not explain misselection

Of the 113 misselected queries at 10^6 rows, the median q-error was 1.017, and 98.2% had q-error below 1.5. The optimiser's row estimates were essentially correct and it selected badly regardless. Figure 1 plots regret against

estimation error and shows none of the relationship the join-ordering literature would predict.

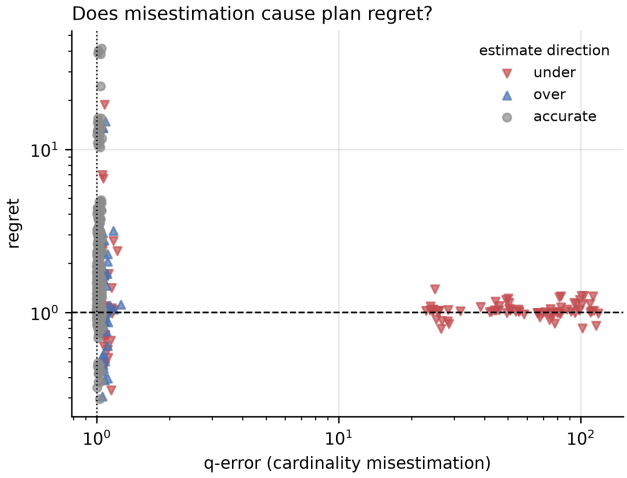

Figure 1: Access-path regret against cardinality estimation error, by direction of the error. High-regret queries cluster at q-error near one, so misestimation does not account for them.

We regard this as the paper's most important negative result. It excludes the explanation that Leis et al. (2015) would suggest and directs attention to the cost model and to path generation. The divergence from their finding is explicable: join ordering has a combinatorially large plan space in which small estimation errors compound, whereas access-path selection has three or four candidates and no compounding, so cost-model calibration and path enumeration dominate instead.

5.3. RQ4a: extended statistics

`CREATE STATISTICS` is the documented remedy for correlated predicates (PostgreSQL Global Development Group, 2024b). It succeeds at its stated objective and degrades the outcome (Table 4, Figure 2).

Mechanism. Without extended statistics the optimiser uses the composite index on (a, b) in a single index scan. With them it switches to a `BitmapAnd` over two single-column indexes. Both plans fetch the identical 7,317 heap blocks, but the second performs two index scans plus a bitmap intersection in place of one scan. The reason for the switch is visible in the node-level estimates:

Table 4: Effect of extended statistics on dependent conjunctive predicates at 10^6 rows. Estimates become nearly exact; every affected query becomes slower.

Dependency	q-error		Est. rows	True	Median ms		Slow-
strength	without	with	with	rows	without	with	down
0.25	25.23	1.11	2,767	2,556	0.93	2.54	2.73×
0.50	49.52	1.04	5,200	5,053	2.12	3.53	1.66×
0.75	75.89	1.02	7,633	7,513	2.80	4.34	1.55×
1.00	113.19	1.05	9,700	9,995	3.95	5.06	1.28×

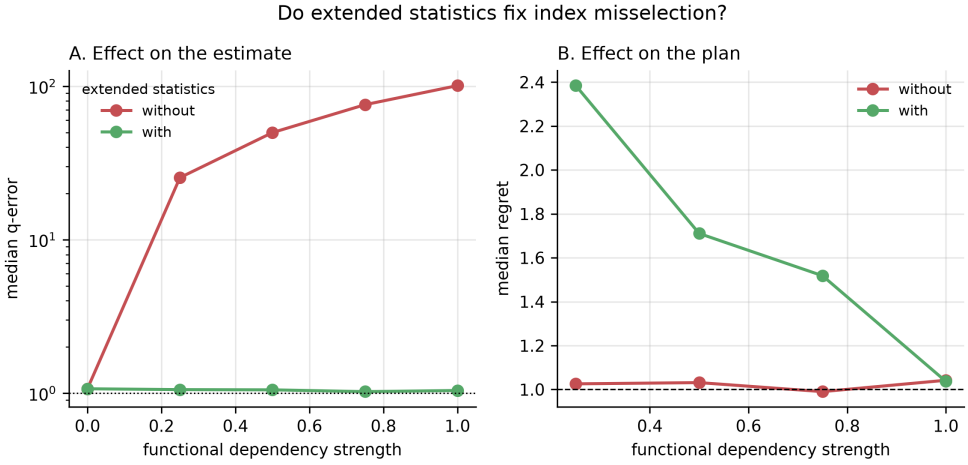

Figure 2: Extended statistics against dependency strength. Panel A: the estimate improves by up to two orders of magnitude. Panel B: the resulting plan degrades. Improving the estimate and improving the plan are separable outcomes.

```
Bitmap Heap Scan  ... rows=10367     <- extended statistics
   applied
  -> BitmapAnd    ... rows=107       <- independence
     assumption, not corrected
```

Listing 1: Node row estimates with extended statistics present.

Since $10367^2/10^6 \approx 107$, the `BitmapAnd` node is still multiplying per-column selectivities under exactly the independence assumption that extended statistics exist to replace. The optimiser consequently prices the `BitmapAnd` at 635 using the uncorrected figure and the composite path at 13,558 using the corrected one, a 21-fold discrepancy, and selects the plan that is in fact slower.

We verified that the optimiser is behaving rationally given its own model by restricting the available indexes. With only the composite index present and extended statistics active, that path is priced at 13,558 yet executes in 4.030 ms, faster than the `BitmapAnd` at 4.794 ms. Without extended statistics the same composite path is priced at 392.80 and executes in 4.155 ms. The corrected estimate therefore makes an equally fast plan appear 34× more expensive. The defect is not the optimiser's choice procedure but the inconsistent propagation of the correction across path types.

5.4. RQ4b: reducing `random_page_cost`

The default `random_page_cost` of 4.0 encodes a magnetic-disk assumption, and guidance widely recommends reducing it on flash storage. Measured across the full query set, the recommendation is counterproductive in our environment (Table 5, Figure 3).

Table 5: Total execution time over the full query set at each `random_page_cost`, conventional index set.

`random_page_cost`	Total time (ms)	Versus best
1.0	2,306	2.3× worse
1.1	1,954	1.9× worse
1.2	1,942	1.9× worse
1.5	1,006	best
2.0	1,053	1.0×
3.0	1,053	1.0×
4.0 (default)	1,068	1.06×

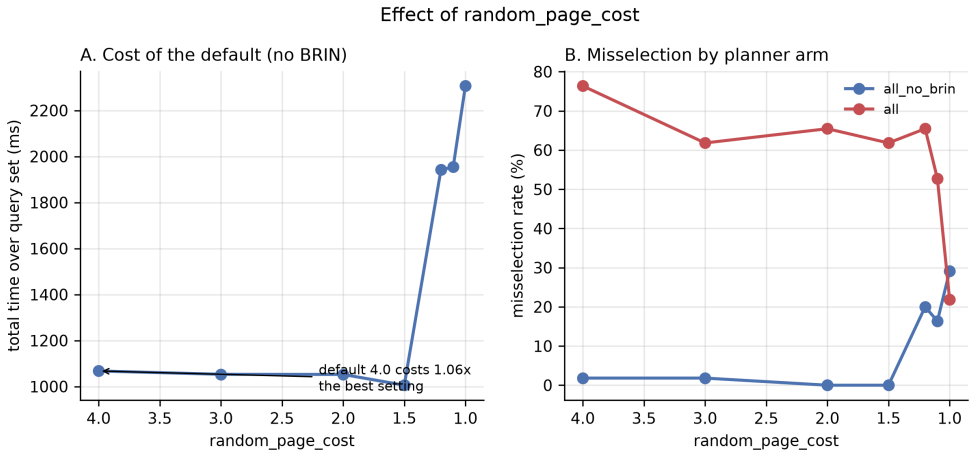

Figure 3: Effect of `random_page_cost`. Panel A: total query-set time; reducing the setting below 1.5 degrades the workload. Panel B: misselection rate by index configuration.

The default lies within 6% of optimal. Reducing it to 1.0 costs a factor of 2.3. Access-path counts explain the mechanism: at `random_page_cost` = 1.0 the optimiser abandons bitmap scans entirely in favour of plain index scans, 136 index scans against zero bitmap scans, which is the wrong choice for higher-selectivity predicates because it forfeits the physical-order sorting that makes bitmap heap access efficient.

A single-query measurement early in this study indicated the default cost 24× (Table 6). That observation was accurate but unrepresentative. We retain it as a documented instance of how anecdotal tuning evidence misleads, a pattern Raasveldt et al. (2018) identify as endemic.

Table 6: Single-query sensitivity to `random_page_cost`, identical row estimate (10,662) throughout. Compare with Table 5.

`random_page_cost`	Plan chosen	Median ms
4.0 (default)	sequential scan	49.14
2.0	B-tree index scan	2.08
1.5	B-tree index scan	2.33
1.1	B-tree index scan	2.49
1.0	B-tree index scan	2.25

5.5. RQ2 and RQ4c: a co-located BRIN index

BRIN indexes are small and cheap to build, so adding one is commonly regarded as harmless. It is not, while the table is buffer-pool resident (Table 7, Figure 4).

Table 7: Effect of adding a BRIN index alongside an existing B-tree on the same column.

Scale	With BRIN	Without BRIN	Median R	Max R
10^6	73.3%	2.3%	2.06 vs 1.01	18.78
10^7	30.1%	32.3%	1.04 vs 1.04	2.63

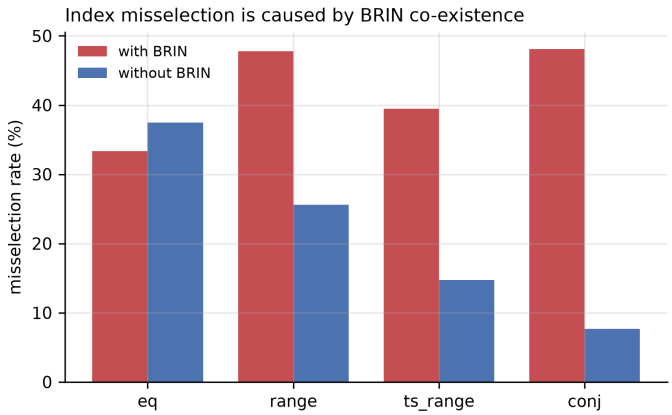

Figure 4: Misselection rate by query family with and without BRIN in the index set at 10^6 rows. The conjunctive family is the control: no BRIN index covers those columns and correspondingly no effect appears.

The conjunctive family serves as an internal control. No BRIN index covers columns a or b, and correspondingly the matched comparison shows a median speedup of 0.90× from removing BRIN for that family, against 2.83× and 2.78× for the range families. The effect appears exactly where the mechanism predicts it should.

Mechanism. With both indexes present and a bitmap plan forced by disabling other access methods, the optimiser selected BRIN in 16 of 16 cases, and in ten of those the B-tree bitmap path was cheaper *by the optimiser's own cost model*, in one case by a factor of nine (Table 8). A cost-based optimiser does not knowingly select a path it prices as nine times more expensive,

which indicates the B-tree bitmap path is eliminated during path generation rather than rejected during costing. We state this as an inference from cost arithmetic; confirming it against the source is future work.

Table 8: Forced bitmap plans with both indexes present, statistics held byte-identical within each row. BRIN is selected in every case.

Corr.	Query	BRIN cost	BRIN ms	B-tree cost	B-tree ms
0.0	0.1% range	29,320	65.95	3,166	0.34
0.0	1% range	29,322	66.73	13,857	5.14
0.5	0.1% range	15,338	66.33	3,174	0.21
0.5	1% range	14,852	63.90	13,848	2.76
0.95	0.1% range	13,511	62.07	3,294	0.09
0.95	1% range	15,365	66.45	14,201	0.76

The execution cost of the selected plan is visible in its profile: `Heap Blocks: lossy=14304` with `Rows Removed by Index Recheck: 990000`. The BRIN path reads essentially the entire table and discards 99% of what it reads. The largest penalty observed is the 0.95-correlation, 0.1% selectivity case: 62.07 ms against 0.09 ms, a factor of 690.

5.6. RQ5: locating the boundaries

Comparing 10^6 against 10^7 rows shows the penalty present at the smaller scale and absent at the larger. Two points, however, cannot distinguish gradual decay from a sharp transition, nor reveal non-monotonic behaviour between them. We therefore measured five intermediate scales (Table 9, Figure 5).

There are two transitions, not one.

Frequency collapses at the buffer pool boundary. The gap in misselection rate falls from 71.4 percentage points at 10^6 rows to 3.4 at 1.25×10^6. That is precisely where the heap crosses `shared_buffers`: 111.8 MB against a 128 MB pool becomes 139.8 MB. The composition of the collapse is instructive. The BRIN-inclusive configuration improves from 71.4% to 37.9%, but the conventional configuration *degrades* from 0.0% to 34.5%. The gap closes principally because the baseline deteriorates once the working set no longer fits, not because BRIN improves.

Table 9: The BRIN penalty across table size. Frequency and severity collapse at different scales.

Million rows	Heap (MB)	Blocks read outside pool	Misselection %		Max regret	
			with BRIN	without	with BRIN	without
1.00	111.8	0	71.4	0.0	18.78	1.06
1.25	139.8	0	37.9	34.5	38.81	1.67
1.50	167.8	0	31.0	17.2	40.32	1.76
2.00	223.2	0	25.8	12.9	38.34	1.77
3.00	335.2	0	30.3	12.1	41.51	1.67
5.00	558.2	1,902	36.4	30.3	6.98	1.81
10.00	1,116.2	89,081	38.2	44.4	1.92	3.70

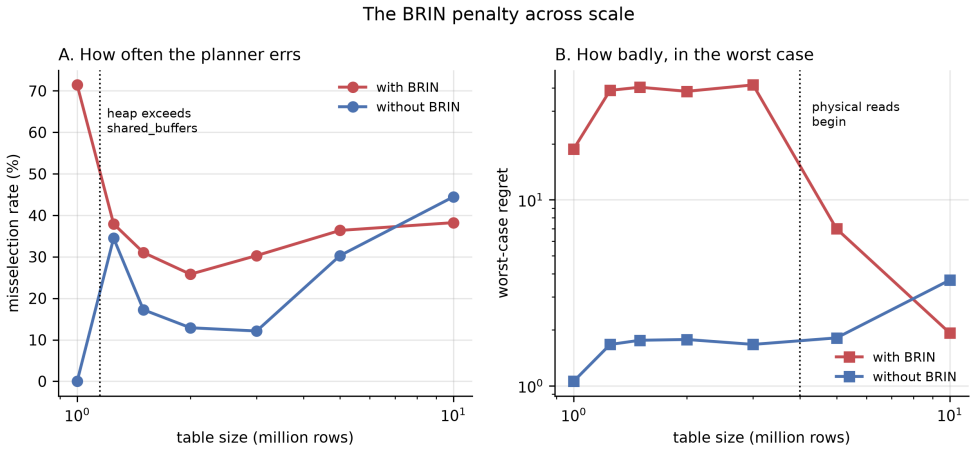

Figure 5: The BRIN penalty across scale. Panel A: how often the optimiser errs. Panel B: worst-case regret. The two collapse at different table sizes, near different system boundaries.

Severity collapses later, after first intensifying. Worst-case regret with BRIN does not decline past the buffer pool boundary. It approximately doubles, from 18.78 at 10^6 rows to 38.81 at 1.25×10^6, and remains near 40 through 3×10^6. It falls only at 5×10^6 (6.98) and 10^7 (1.92). That later collapse coincides with the onset of physical reads: blocks fetched from outside the buffer pool are zero up to 3×10^6 rows, 1,902 at 5×10^6 and 89,081 at 10^7.

The practical implication inverts the naive reading. A deployment whose table is two to three times `shared_buffers` experiences the most severe individual regressions measured anywhere in this study, worse than at the scale where the effect was first identified. Had we reported only the two-point comparison, we would have concluded that the penalty diminishes with scale and implicitly reassured exactly the practitioners most at risk.

We do not claim a verified causal mechanism for either transition. What is established is that each coincides with a specific, independently measurable system boundary, and that the two boundaries are distinct.

5.7. Incidental finding: hash index construction under skew

Hash index build time proved acutely sensitive to value skew (Table 10). Zipfian values collide into few hash buckets, producing long overflow chains, and PostgreSQL does not parallelise hash construction as it does B-tree construction. The practical caution is that hash index creation cannot be assumed cheap on skewed columns, a consideration absent from index-selection tooling that models index benefit but not construction cost under skew (Kossmann et al., 2020).

Table 10: Index construction time at 10^7 rows by value skew.

Dataset	`zipf_s`	Hash build	B-tree build
baseline	0.0	11.6 s	3.2 s
skew05	0.5	16.7 s	2.3 s
skew10	1.0	187.7 s	2.5 s
skew15	1.5	2,152.9 s	2.3 s

6. Discussion

6.1. A common failure pattern

The three practices fail for a shared reason: each optimises a single quantity, while plan quality depends on several.

Extended statistics optimise *estimate accuracy*. A more accurate estimate improves the plan only if it is applied consistently wherever the plan is priced, and Section 5.3 shows it is not. The partial correction is worse than no correction, because it makes two candidate paths priced under different assumptions directly comparable when they are not.

Reducing `random_page_cost` optimises for *storage latency*. The premise is sound: random access on flash is cheaper relative to sequential access than the default assumes (Graefe, 2009). But the same constant governs the choice between bitmap and plain index access, so reducing it improves one decision while degrading another, and Section 5.4 shows the second effect dominates below 1.5.

Adding a BRIN index optimises for *index size and construction cost*, on both of which BRIN genuinely excels. But index availability also determines which paths the optimiser generates, and Section 5.5 shows a cheap index can capture the bitmap path and exclude a superior one.

In each case the advice is correct about the quantity it names and wrong about the outcome. This generalises beyond the three specific cases: mechanism-based tuning advice is systematically vulnerable to this failure whenever the mechanism is real but not the only mechanism in play. The corrective is workload-level measurement, which is exactly what such advice typically lacks.

6.2. Reporting boundaries

Section 5.6 carries a methodological point beyond its specific finding. Our own two-point comparison would have supported a conclusion the intermediate measurements refute, and the refutation was not a matter of degree but of direction: severity increases where the two-point reading suggests it decreases.

We suggest that empirical database studies reporting a performance effect should, where feasible, also report the conditions under which it ceases to hold. Without that, practitioners cannot determine applicability, and apparent contradictions between studies measuring at different scales cannot be distinguished from genuine disagreement.

6.3. Recommendations for practitioners

1. Do not create extended statistics reflexively. Verify that the *plan* improved, not merely the estimate; the two are separable and can move in opposite directions.

2. Do not reduce `random_page_cost` below approximately 1.5 without workload measurement. The default was within 6% of optimal here.
3. Treat a BRIN index co-located with a B-tree on the same column as a risk requiring measurement, with particular attention when the table is between one and three times `shared_buffers`.
4. Do not assume hash index construction is inexpensive on skewed columns.

7. Threats to Validity

Internal validity. Regret is a lower bound, since the denominator is the fastest measured configuration rather than a proven optimum. Values marginally below 1.0 occur because the free-choice configuration can combine indexes in ways no single restricted configuration can, which we report rather than clamp. The misselection threshold of 1.2 is a judgement call; full distributions are published so that alternatives may be applied.

Construct validity. The path-elimination mechanism in Section 5.5 is inferred from cost arithmetic rather than confirmed against PostgreSQL's source code. The extended-statistics result requires a redundant index set, a composite index alongside both single-column indexes, which is common in practice but not universal; where only the composite index exists the effect cannot arise.

External validity. The study covers one DBMS, one version and one hardware configuration. Whether the findings reflect PostgreSQL implementation choices or general properties of cost-based access-path selection is untested, and is the most significant limitation. All measurements are warm-cache; reliably evicting the operating system page cache is not portable, so cold-storage behaviour is out of scope rather than approximated. Because the host has 23.8 GB of memory, the page cache retains the data even at 10^7 rows, so our larger scales test departure from the *buffer pool* rather than from memory, and the two cannot be separated on this hardware. Only single-relation access paths are considered; join ordering is deliberately excluded as a separate and well-studied problem (Leis et al., 2018).

8. Conclusion

PostgreSQL 17's access-path selection is close to optimal under a conventional index set, selecting the best available plan for 97.7% of the queries

measured. Where it fails, cardinality misestimation is not the cause: 98.2% of misselected queries had accurate row estimates, in contrast to the established result for join ordering.

The failures instead trace to cost modelling and path generation, and are triggered by three practices intended to prevent them. Extended statistics improve estimates by up to 108× and slow queries by up to 2.7×, because the correction is not propagated to the `BitmapAnd` node. Reducing `random_page_cost` on flash storage costs a factor of 2.3, the default lying within 6% of optimal. A co-located BRIN index raises misselection from 2.3% to 73.3% and can slow individual queries by 194×.

That last effect has two boundaries rather than one. Its frequency collapses when the heap exceeds `shared_buffers`, while its worst case doubles at that same point and subsides only when queries begin incurring physical reads. The most severe regressions occur at two to three times the buffer pool size, not at the smallest scale, and a two-point comparison would have indicated the opposite.

A tuning recommendation is not correct or incorrect in isolation; it is correct or incorrect under conditions, and those conditions are measurable. Reporting an effect without its boundaries invites practitioners to apply it where it does not hold, or to dismiss it where it does.

Future work. Three directions follow. Confirming the path-elimination mechanism against PostgreSQL's source would convert our strongest inference into a verified claim. Replicating the study on a second DBMS would determine whether these are PostgreSQL characteristics or general properties of cost-based access-path selection, which is the most consequential open question. Extending to hardware permitting controlled page-cache eviction would separate buffer-pool residency from operating-system cache residency, a distinction the present platform cannot make.

Data availability

All source code, raw measurements, generated figures and an executable analysis notebook are publicly available (Sajin, 2026). No dataset is distributed: all data is generated from a fixed seed, so the corpus is byte-identical on any machine. Measurement runs are resumable at the granularity of a single execution, and each record retains all individual repetitions rather than only the median, so variance claims can be verified independently.

CRediT authorship contribution statement

Md. Imtiaj Alam Sajin: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Ashraf Uddin:** Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported in this paper.

References

Boncz, P., Neumann, T., Erling, O., 2013. TPC-H analyzed: Hidden messages and lessons learned from an influential benchmark, in: Performance Characterization and Benchmarking, LNCS 8391. Springer, pp. 61–76. doi:10.1007/978-3-319-04936-6_5.

Chaudhuri, S., 1998. An overview of query optimization in relational systems, in: Proc. ACM Symp. on Principles of Database Systems (PODS), pp. 34–43. doi:10.1145/275487.275492.

Chaudhuri, S., Narasayya, V.R., 1997. An efficient cost-driven index selection tool for Microsoft SQL Server, in: Proc. 23rd Int. Conf. on Very Large Data Bases (VLDB), pp. 146–155.

Comer, D., 1979. The ubiquitous b-tree. ACM Computing Surveys 11, 121–137. doi:10.1145/356770.356776.

Effelsberg, W., Haerder, T., 1984. Principles of database buffer management. ACM Transactions on Database Systems 9, 560–595. doi:10.1145/1994.2022.

Graefe, G., 1993. Query evaluation techniques for large databases. ACM Computing Surveys 25, 73–169. doi:10.1145/152610.152611.

Graefe, G., 2009. The five-minute rule 20 years later, and how flash memory changes the rules. Communications of the ACM 52, 48–59. doi:10.1145/1538788.1538805.

Graefe, G., 2011. Modern b-tree techniques. Foundations and Trends in Databases 3, 203–402. doi:10.1561/1900000028.

Han, Y., Wu, Z., Wu, P., Zhu, R., Yang, J., Tan, L.W., Zeng, K., Cong, G., Qin, Y., Pfadler, A., Qian, Z., Zhou, J., Li, J., Cui, B., 2021. Cardinality estimation in DBMS: A comprehensive benchmark evaluation. Proc. VLDB Endowment 15, 752–765. doi:10.14778/3503585.3503586.

Ioannidis, Y.E., Christodoulakis, S., 1991. On the propagation of errors in the size of join results. ACM SIGMOD Record 20, 268–277. doi:10.1145/119995.115835.

Kossmann, J., Halfpap, S., Jankrift, M., Schlosser, R., 2020. Magic mirror in my hand, which is the best in the land? an experimental evaluation of index selection algorithms. Proc. VLDB Endowment 13, 2382–2395. doi:10.14778/3407790.3407832.

Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N., 2018. The case for learned index structures, in: Proc. ACM SIGMOD Int. Conf. on Management of Data, pp. 489–504. doi:10.1145/3183713.3196909.

Lan, H., Bao, Z., Peng, Y., 2021. A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration. Data Science and Engineering 6, 86–101. doi:10.1007/s41019-020-00149-7.

Lee, S.W., Moon, B., Park, C., 2009. Advances in flash memory SSD technology for enterprise database applications, in: Proc. ACM SIGMOD Int. Conf. on Management of Data, pp. 863–870. doi:10.1145/1559845.1559937.

Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T., 2015. How good are query optimizers, really? Proc. VLDB Endowment 9, 204–215. doi:10.14778/2850583.2850594.

Leis, V., Radke, B., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T., 2018. Query optimization through the looking glass, and what we found running the join order benchmark. The VLDB Journal 27, 643–668. doi:10.1007/s00778-017-0480-7.

Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M., Kraska, T., 2021. Bao: Making learned query optimization practical, in: Proc. ACM SIGMOD Int. Conf. on Management of Data, pp. 1275–1288. doi:10.1145/3448016.3452838.

Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O., Tatbul, N., 2019. Neo: A learned query optimizer. Proc. VLDB Endowment 12, 1705–1718. doi:10.14778/3342263.3342644.

Moerkotte, G., Neumann, T., Steidl, G., 2009. Preventing bad plans by bounding the impact of cardinality estimation errors. Proc. VLDB Endowment 2, 982–993. doi:10.14778/1687627.1687738.

Negi, P., Marcus, R., Kipf, A., Mao, H., Tatbul, N., Kraska, T., Alizadeh, M., 2021. Flow-loss: Learning cardinality estimates that matter. Proc. VLDB Endowment 14, 2019–2032. doi:10.14778/3476249.3476259.

PostgreSQL Global Development Group, 2024a. PostgreSQL 17 documentation, chapter 68: BRIN indexes. https://www.postgresql.org/docs/17/brin.html. Accessed 16 August 2026.

PostgreSQL Global Development Group, 2024b. PostgreSQL 17 documentation: CREATE STATISTICS. https://www.postgresql.org/docs/17/sql-createstatistics.html. Accessed 16 August 2026.

PostgreSQL Global Development Group, 2024c. PostgreSQL 17 documentation, section 14.2: Statistics used by the planner. https://www.postgresql.org/docs/17/planner-stats.html. Accessed 16 August 2026.

PostgreSQL Global Development Group, 2024d. PostgreSQL 17 documentation, section 19.7: Query planning configuration. https://www.postgresql.org/docs/17/runtime-config-query.html. Accessed 16 August 2026.

PostgreSQL Global Development Group, 2024e. PostgreSQL 17 documentation, section 63.6: Index cost estimation functions. https://www.postgresql.org/docs/17/index-cost-estimation.html. Accessed 16 August 2026.

PostgreSQL Hackers Mailing List, 2019. BRIN index which is much faster never chosen by planner. https://www.postgresql.org/message-id/20191015224047.GV3599@telsasoft.com. Accessed 16 August 2026.

Raasveldt, M., Holanda, P., Gubner, T., Muehleisen, H., 2018. Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: Proc. 7th Int. Workshop on Testing Database Systems (DBTest), pp. 1–6. doi:10.1145/3209950.3209955.

Sajin, I., 2026. Access-path selection benchmark for PostgreSQL: Code, raw measurements and analysis. https://github.com/Imtiaj-Sajin/Reseach-on-Database-Architecture. Accessed 16 August 2026.

Selinger, P.G., Astrahan, M.M., Chamberlin, D.D., Lorie, R.A., Price, T.G., 1979. Access path selection in a relational database management system, in: Proc. ACM SIGMOD Int. Conf. on Management of Data, pp. 23–34. doi:10.1145/582095.582099.

Silberschatz, A., Korth, H.F., Sudarshan, S., 2019. Database System Concepts. 7 ed., McGraw-Hill.

Stonebraker, M., Cetintemel, U., 2005. One size fits all: An idea whose time has come and gone, in: Proc. 21st Int. Conf. on Data Engineering (ICDE), pp. 2–11. doi:10.1109/ICDE.2005.1.

Wongkham, C., Lu, B., Liu, C., Zhong, Z., Lo, E., Wang, T., 2022. Are updatable learned indexes ready? Proc. VLDB Endowment 15, 3004–3017. doi:10.14778/3551793.3551848.

Wu, W., Chi, Y., Zhu, S., Tatemura, J., Hacigumus, H., Naughton, J.F., 2013. Predicting query execution time: Are optimizer cost models really unusable?, in: Proc. IEEE 29th Int. Conf. on Data Engineering (ICDE), pp. 1081–1092. doi:10.1109/ICDE.2013.6544899.